
Good Agentic Friends Do Not Just Give Verbal Advice: They Can Update Your Weights

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Multi-agent LLM systems usually collaborate by exchanging natural-language
2 messages. This interface is simple and interpretable, but it forces each sender’s
3 intermediate computation to be serialized into tokens and then reprocessed by
4 the receiver, thereby increasing the generated-token cost, prefill overhead, and
5 KV-cache memory. We study an alternative communication interface: instead of
6 appending a sender’s message to the receiver’s context, compile the sender’s hidden
7 states into a transient, receiver-specific weight perturbation. We introduce **TFLOW**
8 (Thought Flow), a weight-space communication framework for a known and fixed
9 receiver architecture. For each query, frozen role-prompted sender agents process
10 the input, and a learned parameter generator maps their internal activations into
11 low-rank LoRA perturbations targeting the receiver’s modules. These perturbations
12 are fused and applied only during the receiver’s generation, enabling instance-level
13 adaptation without permanently changing the model or enlarging the receiver’s text
14 context. With three Qwen3-4B agents, **TFLOW** improves over a standalone receiver
15 by up to 8.5 accuracy points across five benchmarks while reducing processed
16 tokens by up to 32.69%. Compared with a text-based three-agent baseline, it
17 reduces total processed tokens by up to 83.27% and the wall-clock inference time
18 by up to 4.6×, while maintaining competitive accuracy on four of five benchmarks.
19 These results suggest that transient low-rank weight perturbations can serve as an
20 executable communication medium for efficient multi-agent LLM collaboration.

21 1 Introduction

22 Multi-agent LLM systems are often built like small committees. One model proposes a strategy,
23 another contributes domain knowledge, and a final model produces the answer. This design has been
24 effective across reasoning, coding, and tool-use settings [1, 2, 3, 4, 5, 6, 7]. Yet the communication
25 interface between agents has changed surprisingly little: most systems still require each agent to
26 write a natural-language message that is appended to another agent’s context.

27 This text interface is attractive because it is universal and human-readable. **We yet argue it might**
28 **be an awkward interface between neural networks.** A sender agent has already transformed the
29 input into a rich internal state, but conventional multi-agent systems force this state to be decoded
30 into tokens before the receiver can use it. The receiver must then encode those tokens again, paying
31 additional prefill cost and storing a larger KV cache (Figure 1 (i)), whose memory and latency grow
32 with context length [8, 9]. As the number of agents or communication rounds grows, this repeated
33 write-read cycle can dominate the cost of collaboration. The issue is not only that natural-language
34 messages are long; it is that they make model-to-model communication pass through a channel
35 designed for humans.

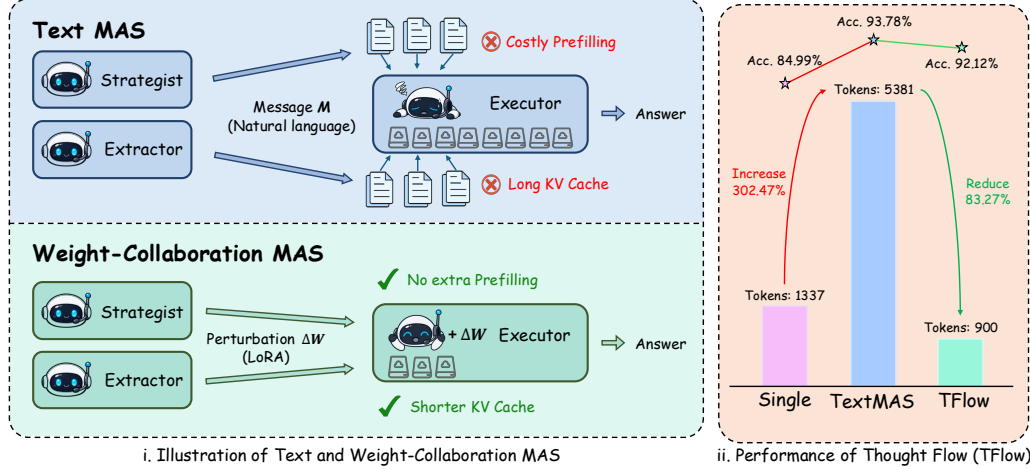


Figure 1: (i) Comparison between Text-based MAS and the proposed Weight-Collaboration MAS. In Text MAS, auxiliary agents transmit natural language messages to the Executor, incurring costly prefilling overhead and inflated KV cache. In contrast, our proposed paradigm compresses inter-agent communication into lightweight LoRA weight perturbations ΔW , which are directly merged into the parameters, thereby eliminating the extra prefilling and significantly reducing the KV cache footprint. (ii) Performance overview on GSM8K. TFlow achieves accuracy competitive with TextMAS while reducing token consumption by 76.7%, substantially surpassing the single-agent baseline in both accuracy and efficiency.

Recent work has begun to question this design by allowing agents to exchange continuous representations rather than text [10, 11, 12, 13, 14, 15]. These latent-space protocols reduce decoding overhead and can preserve information that would be difficult to express in words. However, they still require the receiver to consume sender information as activations, embeddings, or cache states. This creates a different compatibility problem: the transmitted object must be meaningful inside the receiver’s representation geometry, making such methods most natural when agents share architectures, hidden spaces, or carefully trained adapters.

We explore a complementary route. *Instead of sending information that the receiver must read, we send information that changes how the receiver computes.* In particular, we ask whether a sender’s internal state can be compiled into a temporary low-rank perturbation of the receiver’s weights [16, 17]. The receiver then solves the query under this perturbed computation, after which the perturbation is removed, and the frozen base model is restored. In this view, the communication object is not a sentence and not a hidden state injected into the receiver’s cache. It is a query-specific update in the receiver’s parameter space.

We instantiate this idea in **TFLOW** (Thought Flow), a framework for weight-space communication among LLM agents. We focus on a practical fixed-receiver setting: the receiver architecture and target modules are known, all LLM backbones remain frozen, and only a receiver-specific parameter generator [18, 19] is trained. Given a query, each sender agent processes the prompted input once and exposes its hidden states as a conditioning signal. The parameter generator maps this signal into layer- and module-specific LoRA factors for the receiver. Perturbations from multiple senders are fused and injected by transiently patching the forward pass of the receiver’s target linear layers. The receiver then generates its answer without seeing any sender-written message in its text context (Figure 1 (ii)).

This design separates TFLOW from both text-based and latent-space collaboration. Compared with text-based MAS, TFLOW avoids expanding the receiver’s input with auxiliary messages, thereby reducing generated-token usage, prefill overhead, and KV-cache growth. Compared with latent-state transfer, TFLOW does not ask the receiver to directly interpret a sender’s activations. Sender states are instead translated by a learned generator into perturbations that are compatible with the receiver’s own weight tensors by construction. The result is an executable communication channel: sender information affects the receiver by modifying the computation path used for the current query.

We evaluate TFLOW in a three-agent setting with a frozen Qwen3-4B backbone, where sender agents provide different perspectives through hidden states and the receiver produces the final answer. Across five benchmarks, TFLOW improves over the standalone receiver by up to 8.5 accuracy points while

reducing total processed tokens by up to 32.69%. Relative to a text-based three-agent baseline, it cuts processed tokens by up to 83.27% and wall-clock inference time by up to 4.6x, with competitive accuracy on four of five benchmarks; the remaining gap on HumanEval+ suggests that additional generation budget matters more than inter-agent communication for code synthesis.

Our contributions are:

- Motivated by the high communication cost of text-based multi-agent systems, we propose *weight-space communication*, a new communication paradigm for multi-agent LLM collaboration, in which sender information is transmitted as transient low-rank perturbations to a frozen receiver model rather than as text appended to the receiver’s context.
- Under this paradigm, we introduce **TFLOW**, a receiver-specific framework that transforms sender hidden states into query-specific LoRA factors, integrates multiple sender contributions, and injects the resulting perturbation only during the receiver’s generation.
- We demonstrate that TFLOW achieves substantial efficiency gains across five reasoning, knowledge, and coding benchmarks, dramatically reducing generated-token and latency costs relative to text-based multi-agent collaboration with only negligible performance degradation, while consistently outperforming a standalone receiver.

2 Related Work

2.1 LLM-Based Multi-Agent Systems

Multi-Agent Collaboration Paradigms. Large language models have catalyzed a wave of multi-agent systems. Early explorations such as CAMEL [1] and Generative Agents [20] demonstrate the power of role-playing dialogue, enabling agents to cooperate through structured natural-language conversations. Subsequent work applies multi-agent collaboration to complex engineering workflows: MetaGPT [2] and ChatDev [21] mirror real software companies by assigning specialized roles and routing structured artifacts between them, while AutoGen [3] and AutoAgents [22] provide general-purpose frameworks for orchestrating and automatically generating task-specific agents. Another work leverages multi-agent debate to improve reasoning quality [4, 5]. ReConcile [23] organizes round-table conferences among diverse LLM families with confidence-weighted voting. More recently, Mixture-of-Agents [24] and [25] show that iteratively aggregating or even naively scaling diverse model outputs can surpass the strongest individual model. Despite their architectural diversity, all of the above systems share a common reliance on natural language as the primary medium for inter-agent communication.

Inter-Agent Communication Mechanisms. The communication mechanism is central to the effectiveness of any multi-agent system. The prevailing paradigm transmits information as natural-language messages concatenated into each receiver’s context window [1, 2, 3, 21]. While natural language provides a universal and human-interpretable interface, serializing an agent’s intermediate computation into tokens may lose information that is difficult to verbalize and can introduce substantial communication overhead [26, 13, 4, 27]. Recent work has therefore explored non-textual communication channels: CIPHER [26] lets agents communicate through probability-weighted output embeddings; Interlat [12] transmits hidden states as a compressed representation of an agent’s internal computation; and [13, 28, 14] further formalize and extend latent-space communication. These approaches show that moving beyond natural-language messages can preserve richer intermediate signals and reduce token-level overhead, primarily by communicating in activation or embedding space. In this work, we explore a complementary paradigm: *weight-space communication*, where sender-side computation is converted into targeted, instance-specific perturbations of the receiver’s model parameters, which guide the receiver’s generation.

2.2 Knowledge Representation and Operations in Weight Space

Weight-Space Structure and Static Merging. A growing body of evidence shows that neural network weight spaces are structured carriers of transferable knowledge [29, 30]. The *task vector* framework [31] demonstrates that the element-wise difference between a fine-tuned model and its pretrained initialization encodes the learned task skill. Building on this insight, a family of

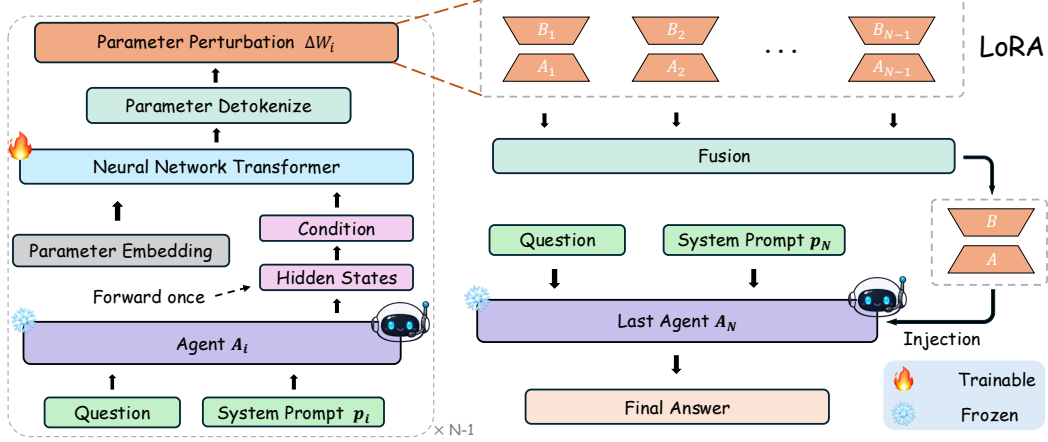


Figure 2: **Overview of TFLOW.** TFLOW realizes multi-agent collaboration through dynamic, instance-specific parameter generation. Given an input question, each sender agent A_i receives the question together with its system prompt p_i and performs a single frozen forward pass to produce hidden-state representations that summarize its role-specific reasoning signal. These hidden states serve as conditions for a trainable **parameter generator**, which maps them into parameter embeddings, applies a neural transformation, and detokenizes the generated representation into a low-rank perturbation ΔW_i , parameterized by **LoRA factors** (A_i, B_i). The per-sender LoRA factors from all $N - 1$ agents are subsequently combined by a fusion module to form a unified update ΔW . This perturbation is transiently injected into the frozen receiver agent A_N , which is conditioned on the original question and its own system prompt p_N to generate the final answer. The injected parameters are created on demand for each input and are discarded after generation, leaving the receiver parameters unchanged. During training, all agents remain **frozen** and only the parameter generator is **trainable**.

model merging methods, including Model Soups [32], Fisher merging [33], TIES-Merging [34], and DARE [35], seeks to combine the knowledge of multiple fine-tuned models directly in weight space without additional training. Despite these advances, all of the above approaches operate in a *static* manner, in which the merging coefficients or perturbation directions are fixed once determined and do not adapt to individual inputs at inference time.

Low-Rank Adaptation and Dynamic Weight Generation. LoRAHub [36] consists of multiple LoRA modules with learned scalar weights for generalizing the cross-task, while LoRAs composition [37] and LoRA-Flow [38] further enable selection of dynamic adapters at the instance-level and layer-wise, respectively. Going one step further, recent approaches use hypernetworks [19] to directly *generate* adapter parameters conditioned on input context—from task embeddings [18], to few-shot demonstrations [39], to full documents [40], which eliminates the need for pre-trained adapter libraries.

Our framework, TFLOW, extends this progression. Rather than conditioning on static documents or task embeddings, TFLOW uses a parameter generator to translate the *dynamic reasoning states* of multiple sender agents into low-rank perturbations of a receiver model’s weights.

3 TFLOW: Thought Flow

Figure 2 illustrates the overall pipeline of TFLOW. This section formalizes the setup (Section 3.1), describes how sender hidden states are aggregated into query-dependent conditioning signals (Section 3.2), introduces the parameter generator that emits LoRA factors (Section 3.3), and details the transient injection into the receiver (Section 3.4).

3.1 Problem Setup of Weight-Collaboration MAS

We consider a collaboration of N agents $\{A_1, \dots, A_N\}$ that share a single frozen backbone f_θ but are individuated by role-specific system prompts $\{p_1, \dots, p_N\}$. Given a query q , agents $A_1, \dots, A_{N-1}$

act as *senders* and A_N as the *receiver* [1, 3, 2]. The senders contribute only their hidden-state trajectories, while the receiver is the one who actually decodes the answer. Rather than exchanging tokens, the senders induce, for this single query, a transient parameter displacement applied to a designated set of linear modules inside the receiver:

$$\hat{\theta} = \theta + \Delta \mathbf{W}(q, \{p_i\}_{i=1}^{N-1}; \psi). \quad (1)$$

Here ψ denotes the parameters of a learned generator and is the *only* set of weights updated during training. The backbone tensors themselves are never overwritten: the perturbation is implemented as a temporary forward patch and removed immediately after the receiver call, so different queries always observe the same original backbone.

Two design constraints follow directly and will shape every component below. First, because we rely on the existence of beneficial neighbors rather than on aggressive fine-tuning, $\Delta \mathbf{W}$ must remain a *low-rank, small-norm* displacement. Second, because useful perturbation directions are *instance-specific*, the generator must be conditioned on representations that depend on q itself, not merely on the static role descriptors p_i . We therefore condition on the senders’ hidden states over the prompted input, which simultaneously encodes the role p_i and the content of the query.

Notation. In the remainder of this section, we restrict weight-space perturbations to L selected decoder layers, with M linear modules perturbed in each selected layer. For the m -th perturbed module in the l -th selected layer, we denote its frozen weight matrix by $\mathbf{W}_m^{(l)}$.

3.2 Sender Conditioning

Each sender A_i encodes the prompted input (p_i, q) using the frozen backbone in a single forward pass. We retain the intermediate activations and denote the resulting hidden states at layer l by $\mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}$, for $l = 0, 1, \dots, L_{\text{total}}$.

Learnable layer aggregation. Instead of relying on a single-layer representation, we aggregate each sender’s hidden states across layers using temperature-scaled softmax weights over learnable layer-wise scalars. The resulting conditioning representation $\mathbf{C}_i \in \mathbb{R}^{T_i \times d}$ enables the generator to exploit features at different levels of abstraction while preserving a fixed conditioning dimensionality.

3.3 Parameter Generator

The parameter generator $\mathcal{H}_\psi$ maps each sender conditioning representation $\mathbf{C}_i$ to a complete set of LoRA factors spanning all $L \times M$ targeted modules in the receiver. The generator comprises three stages: conditioning-driven initialization, a multi-axis Transformer trunk, and detokenization into LoRA matrices.

Stage 1: Initialization. We construct a token grid of shape $(L, H \times W, d_{\text{pg}})$, where $H = H_A + H_B$ divides tokens into **A**- and **B**-generating groups, and W spans the target modules and rank slots. Learnable grid queries $\mathbf{Q}_{\text{grid}} \in \mathbb{R}^{L \times HW \times d_{\text{pg}}}$ are initialized by cross-attending to the projected conditioning representation $\text{Proj}_{\text{in}}(\mathbf{C}_i)$. We then add layer-index and module-rank positional embeddings to encode each token’s structural location within the receiver.

Stage 2: Neural Network Transformer. Our generator architecture follows the structured parameter tokenization paradigm introduced by HY-WU [41]. The initialized grid is refined through N_{pg} stacked blocks, each applying three residual attention operations along complementary axes, followed by a feed-forward network. Cross-layer self-attention (SA_L), batched over the HW axis, captures dependencies across receiver layers; intra-layer self-attention (SA_{HW}), batched over the L axis, propagates information across modules and rank slots within each layer; and conditioning cross-attention (CA) re-anchors the generation process to the sender representation. All attention operations employ RoPE [42] along their active sequence axis.

Stage 3: Detokenization. After N_{pg} blocks, the refined grid is partitioned along the H dimension into **A**- and **B**-tokens. Two linear heads map these tokens to parameter slices, which are rearranged into per-sender LoRA factors $\mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $\mathbf{B}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}} \times r}$. The generator is shared across all senders; sender-specific factors are induced solely by the conditioning representation $\mathbf{C}_i$.

188 3.4 Transient Injection

189 The generated factors are injected into each targeted module of the receiver as a low-rank weight
 190 offset. When multiple senders are active, their generated LoRA updates are fused by a lightweight
 191 scalar gate before being applied to the receiver:

$$\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i)) \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}. \quad (2)$$

192 The resulting $\Delta \mathbf{W}_m^{(l)}$ is injected only into the designated receiver module m at layer l . During the
 193 receiver’s forward pass, the frozen linear map is evaluated with this transient additive update. After
 194 generation, the patch is discarded, ensuring that each input induces its own temporary parameterization
 195 and that all subsequent inputs start from the same frozen backbone.

196 3.5 Training Objective

197 **Diversity regulariser.** To prevent the generator from collapsing toward a query-independent LoRA,
 198 we penalise the squared cosine similarity $\mathcal{L}_{\text{div}}(\Theta; x_q) = \cos^2(v(x_q), v_{\text{prev}}^{(\sigma(x_q))})$ between the flattened
 199 LoRA vector $v(x_q)$ of the current sample and the cached vector $v_{\text{prev}}^{(\sigma)}$ from the previous step. This
 200 loss is minimised at orthogonality, equally penalises parallel and anti-parallel collapse, and incurs no
 201 additional forward cost since the cache is updated via stop-gradient after each backward pass.

202 **End-to-end optimization.** The task loss $\mathcal{L}_{\text{task}}$ is the standard masked next-token cross-entropy
 203 computed only over completion tokens, with the receiver conditioned on both the chat-templated
 204 prompt and the injected ΔW . The full training objective combines it with the diversity regulariser:

$$\mathcal{L}(\Theta) = \mathbb{E}_{(x_q, t, \sigma) \sim \mathcal{D}} [\mathcal{L}_{\text{task}}(\Theta; x_q, t) + \lambda_{\text{div}} \mathcal{L}_{\text{div}}(\Theta; x_q)]. \quad (3)$$

205 The trainable parameter set Θ encompasses all components of the parameter generator $\mathcal{H}_\psi$, the
 206 conditioning projection layers, and the gating head; the receiver and sender backbones share the
 207 frozen parameters θ and remain entirely frozen throughout training. We do not add auxiliary weight-
 208 norm or inter-sender cosine penalties in our default objective; empirically, the cache-based diversity
 209 regularizer is sufficient to encourage instance-level specialization.

210 4 Experiments

211 4.1 Experimental Setups

212 **Datasets and evaluation.** We evaluate on five benchmarks spanning mathematical reasoning,
 213 code generation, and multi-disciplinary knowledge: GSM8K [43], MATH [44], MMLU [45, 46],
 214 HumanEval+ [47, 48], and MBPP+ [47, 49]. All benchmarks are evaluated under the zero-shot,
 215 chain-of-thought setting with no in-context exemplars. At inference time, all methods use identical
 216 decoding hyperparameters, which is temperature 0.6, top- p 0.95.

217 **Backbone and baselines.** All methods share a single frozen **Qwen3-4B** [50] backbone, whose
 218 parameters are never updated during parameter generator training or inference. We compare against
 219 three baselines: (i) **Single-Agent**, which directly prompts Qwen3-4B with a task-level system prompt
 220 and represents the backbone’s intrinsic capability; (ii) **TextMAS**, a three-agent text-level collaboration
 221 in which Agents A and B produce strategy and domain-knowledge passages that are concatenated
 222 into Agent C’s context, representing conventional multi-agent cooperation.

223 **Parameter generator configuration.** The system deploys $N = 3$ agents sharing the same backbone
 224 instance: Agent A analyzes reasoning types and plans chain-of-thought structure, Agent B retrieves
 225 domain knowledge and implicit constraints, and Agent C solves the problem after the generated
 226 LoRA perturbations are injected. Specific system prompts are designed for each role. The parameter
 227 generator is a lightweight Transformer ($d_{\text{pg}} = 1024$, head dimension 128, $N_{\text{pg}} = 2$ blocks, token
 228 dimension 256, dimension-accumulation factor $c = 2$) that produces LoRA factors of rank $r = 4$ with
 229 scaling $\alpha = 8$ ($s = \alpha/r = 2$).

Table 1: **Main results across five benchmarks.** For each task, we report accuracy (**Acc.** $\uparrow$, %), average total processed tokens (**Token** $\downarrow$), and end-to-end wall-clock inference time (**Speed**, in seconds). Single, TextMAS, and TFLOW are shown side by side. The **Improvement** columns compare TFLOW with each baseline: $\uparrow/\downarrow$ denote accuracy increases/decreases in absolute points, $\downarrow$ denotes relative token reduction, and $\times$ denotes the speed ratio. **Green** indicates favorable changes, while **red** indicates unfavorable changes.

Tasks	Metrics	Baselines		TFLOW	Improvement	
		Single	TextMAS		Δ Single	Δ TextMAS
General Task						
MMLU-Redux	Acc. $\uparrow$	58.99	71.50	66.97	$\uparrow$ 7.98	$\downarrow$ 4.53
	Token $\downarrow$	1079	4825	998	$\downarrow$ 7.51%	$\downarrow$ 79.32%
	Speed (s)	8226	36450	9784	$\times$ 0.84	$\times$ 3.73
Math & STEM Task						
GSM8K	Acc. $\uparrow$	84.99	93.78	92.12	$\uparrow$ 7.13	$\downarrow$ 1.66
	Token $\downarrow$	1337	5381	900	$\downarrow$ 32.69%	$\downarrow$ 83.27%
	Speed (s)	6230	27256	5953	$\times$ 1.05	$\times$ 4.58
MATH	Acc. $\uparrow$	16.18	26.47	23.16	$\uparrow$ 7.98	$\downarrow$ 3.31
	Token $\downarrow$	2782	8188	2242	$\downarrow$ 19.41%	$\downarrow$ 72.62%
	Speed (s)	1984	5213	2258	$\times$ 0.88	$\times$ 2.31
Coding Task						
MBPP+	Acc. $\uparrow$	59.79	68.52	67.20	$\uparrow$ 7.41	$\downarrow$ 1.32
	Token $\downarrow$	1533	5500	1301	$\downarrow$ 15.13%	$\downarrow$ 76.35%
	Speed (s)	1998	5796	2395	$\times$ 0.83	$\times$ 2.42
Humaneval+	Acc. $\uparrow$	56.71	75.00	65.24	$\uparrow$ 8.53	$\downarrow$ 9.76
	Token $\downarrow$	1756	5879	1662	$\downarrow$ 5.35%	$\downarrow$ 71.73%
	Speed (s)	872	2512	1065	$\times$ 0.82	$\times$ 2.36

Training details. The training data consist of OpenThoughts [51], Sky-T1 [52], and KodCode [53]. We train for one epoch on 32,000 samples using a single NVIDIA RTX PRO 6000, which takes approximately 8 hours.

4.2 Main Results

Table 1 reports accuracy, average total processed token usage, and wall-clock inference time for TFLOW, the Single-Agent baseline, and TextMAS across all five benchmarks.

Accuracy improvement over Single-Agent. TFLOW consistently outperforms the single-agent baseline across all task categories, with accuracy gains of +7.13 to +8.53 points. Notably, these gains come with *fewer* total processed tokens on every benchmark, indicating that weight-space perturbations guide the receiver toward more concise reasoning paths rather than verbose exploration. TFLOW incurs higher end-to-end latency on four of the five benchmarks, because applying instance-specific LoRA perturbations to the backbone prevents efficient batched generation.

Efficiency advantage over TextMAS. As shown in Table 1, total processed-token consumption is cut by 71–83% across all five benchmarks, translating to 2.3–4.6 $\times$ wall-clock speed-ups. These gains stem from weight-space injection, entirely bypassing the lengthy prefilling and KV-cache overhead inherent in text-based context concatenation. Compared with TextMAS, the accuracy gap remains modest—within 1.3–4.5 points on four of the five tasks. The sole outlier is HumanEval+ ($\Delta = 9.76$). We attribute this to TextMAS’s concatenated context serving a dual role: beyond transferring inter-agent knowledge, it implicitly extends the generation budget, enabling longer outputs that facilitate structural elaboration and self-debugging. TFLOW successfully transfers collaborative knowledge through the weight space but does not alter the output distribution’s length characteristics, a distinction that surfaces most clearly in code generation.

4.3 Instance-level TFLOW Analysis

A core property of TFLOW is that the LoRA consumed by the receiver is generated *per input*, rather than learned once and reused. We verify this by examining two points of the pipeline: the conditioning vector $\mathbf{c}$ that drives the parameter generator (section 4.3.1), and the resulting LoRA tensors for the receiver (section 4.3.2).

4.3.1 Hidden-State Evidence

We extract per-layer hidden states for 20 instances per dataset and report mean pairwise cosine similarity in three regimes: within-GSM8K, within-MBPP+, and cross-task (GSM8K $\times$ MBPP+).

Learned layer weights. As shown in Figure 3, the learned weights $\{w_\ell\}$ do *not* simply track the lowest pairwise-similarity layers. More than 80% of the mass concentrates on a small set of layers (L05, L29, L30), each with a distinct similarity profile. This selection emerges purely from end-to-end training, suggesting that these layers are chosen for their conditioning utility rather than surface-level diversity.

Pairwise similarity across regimes. Figure 3 further shows that the aggregated conditioning vector preserves the expected ordering of semantic similarity: within-GSM8K pairs (0.94) are most similar, followed by within-MBPP+ pairs (0.83), with cross-task pairs (0.73) being the least similar. This confirms that the conditioning representation encodes instance-level variation in a structured manner.

Per-layer hidden states mean pairwise cosine similarity

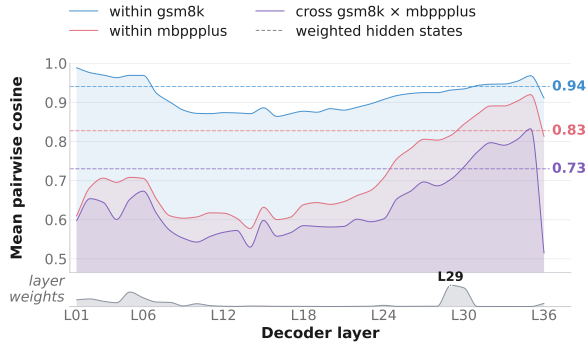


Figure 3: Per-layer mean pairwise cosine similarity of the question’s last-token hidden state across decoder layers. Solid curves report within-task similarity and cross-task similarity, respectively. Dashed lines indicate the corresponding similarity computed on the aggregated conditioning vector $\mathbf{c}$ that the parameter generator actually consumes. The bottom panel shows the learned layer weights $\{w_\ell\}$, which peak sharply at layer 29.

4.3.2 LoRA Tensor Evidence

For each instance, we materialise the effective delta-weights $\Delta W_k = B_k A_k$ across all $K=252$ adapted projections (attention q, k, v, o and MLP up, gate, down on every decoder layer) and concatenate them into a single gauge-invariant fingerprint θ . We then compute the mean pairwise cosine similarity of θ across $N=20$ instances on five benchmarks spanning three domains. Table 2 reports the resulting similarity matrix.

Distinct LoRAs per instance. The within-dataset similarity ranges from 0.264 to 0.933, reflecting the heterogeneity of the underlying instances: Python-coding tasks vary widely in specification, whereas olympiad-style MINERVA-MATH problems share a highly templated structure. This spread indicates that TFLOW naturally allocates more adapter variation to datasets with diverse instances and less to near-templated ones. This is exactly the behaviour one would expect of an architecture that genuinely conditions on the input rather than collapsing to a fixed per-task adapter.

Domain-aware geometry. The off-diagonal entries exhibit clear block structure: within-domain pairs consistently show higher similarity than cross-domain pairs. The math block and the code block are the two highest cross-dataset entries, while the math $\times$ code crossings are the lowest. Notably, the within-dataset similarity of MBPP+ nearly coincides with its cross-similarity to HUMANEVAL+, which is expected given that both are Python function-level benchmarks with overlapping prompt formats. In all cases the diagonal dominates its row, confirming that θ encodes task semantics faithfully.

4.4 Ablation Study: Instance-Specificity of Weight Perturbations

A key design choice in TFLOW is to generate *instance-specific* perturbations: the LoRA factors injected into the receiver are conditioned on both the sender’s reasoning and the particular input.

Table 2: Mean pairwise cosine similarity of the gauge-invariant LoRA fingerprint θ ($N=20$ instances per dataset). Diagonal: within-dataset; off-diagonal: cross-dataset.

	GSM8K	MINERVA-MATH	MMLU	MBPP+	HUMANEVAL+
GSM8K	0.519	0.267	0.148	0.133	0.094
MINERVA-MATH	—	0.933	0.215	0.187	0.220
MMLU	—	—	0.424	0.063	0.099
MBPP+	—	—	—	0.264	0.252
HUMANEVAL+	—	—	—	—	0.315

To validate that this instance-level conditioning is essential to the observed gains, we conduct two ablations that progressively degrade specificity while holding all other variables constant.

Static LoRA. To isolate instance-conditioned parameter generation from standard parameter-efficient adaptation, we replace the parameter generator of TFLOW with a conventional LoRA adapter shared across all inputs, while keeping the backbone, adapted modules, LoRA rank, and training data unchanged.

As shown in Figure 4, Static-LoRA consistently improves over the Single-Agent baseline on all benchmarks. However, TFLOW achieves substantially stronger performance, outperforming Static-LoRA by 4.29 points on average, with especially clear gains on more challenging reasoning and code-oriented benchmarks such as MBPP+ and HumanEval+. This shows that TFLOW’s advantage goes beyond added trainable capacity and stems from input-dependent modulation of the receiver.

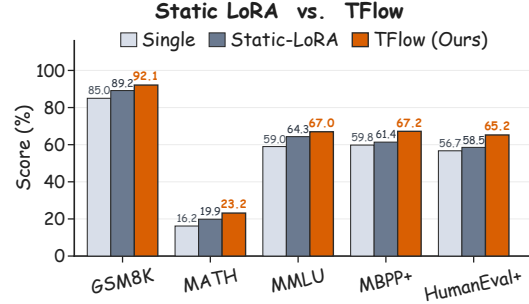


Figure 4: **Static LoRA vs. TFLOW** performance.

Mismatched Perturbation Injection. We further investigate whether TFLOW perturbations encode instance-level information by fixing the receiver input and replacing its matched perturbation with LoRA factors from random sources, other tasks, same-task samples, or the matched sample.

Figure 5 shows that random LoRA perturbations bring only marginal gains, while cross-task perturbations still improve over the baseline, indicating that TFLOW captures transferable collaborative signals. However, same-task perturbations perform better, and the matched-sample perturbation achieves the highest accuracy, suggesting that TFLOW encodes not only task-level knowledge but also fine-grained instance-specific cues. This confirms that the generated perturbations are meaningfully tied to the target input, rather than serving as generic task-level or adapter-like updates.

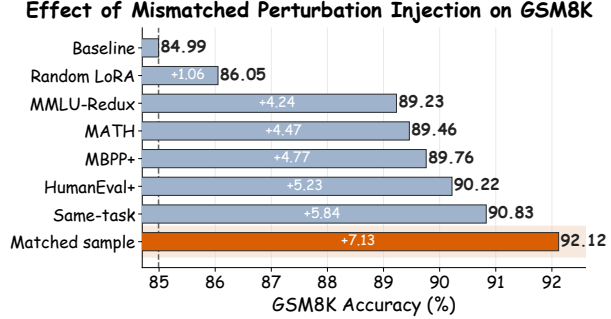


Figure 5: **Mismatched perturbation injection** on GSM8K.

5 Conclusion

We presented TFLOW, a weight-space collaboration paradigm for multi-agent LLM systems. Rather than routing inter-agent knowledge through natural language, TFLOW maps sender agents’ hidden states into query-specific low-rank perturbations of a frozen receiver model. These perturbations are fused and transiently injected at inference time, enabling instance-level adaptation without increasing context length or requiring latent-space compatibility. Across five benchmarks, TFLOW consistently outperforms the single-agent baseline and achieves competitive accuracy with text-based multi-agent systems while using substantially fewer tokens and lower latency. Our findings indicate that model weights can serve as an effective communication medium among agents, suggesting a promising direction for efficient and scalable multi-agent LLM collaboration.

References

- [1] Li, G., H. Hammoud, H. Itani, et al. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.
- [2] Hong, S., M. Zhuge, J. Chen, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*. 2023.
- [3] Wu, Q., G. Bansal, J. Zhang, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*. 2024.
- [4] Du, Y., S. Li, A. Torralba, et al. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*. 2024.
- [5] Liang, T., Z. He, W. Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 17889–17904. 2024.
- [6] Guo, T., X. Chen, Y. Wang, et al. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [7] Xi, Z., W. Chen, X. Guo, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [8] Pope, R., S. Douglas, A. Chowdhery, et al. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.
- [9] Kwon, W., Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626. 2023.
- [10] Li, X. L., P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597. 2021.
- [11] Lester, B., R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 3045–3059. 2021.
- [12] Du, Z., R. Wang, H. Bai, et al. Enabling agents to communicate entirely in latent space. *arXiv preprint arXiv:2511.09149*, 2025.
- [13] Zheng, Y., Z. Zhao, Z. Li, et al. Thought communication in multiagent collaboration. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.
- [14] Zou, J., X. Yang, R. Qiu, et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.
- [15] Jin, H., K. Peng, Y. Yu, et al. Agent primitives: Reusable latent building blocks for multi-agent systems. *arXiv preprint arXiv:2602.03695*, 2026.
- [16] Houlsby, N., A. Giurui, S. Jastrzebski, et al. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR, 2019.
- [17] Hu, E. J., Y. Shen, P. Wallis, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [18] Mahabadi, R. K., S. Ruder, M. Dehghani, et al. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 565–576. 2021.
- [19] Ha, D., A. Dai, Q. V. Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.

- [20] Park, J. S., J. O’Brien, C. J. Cai, et al. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22. 2023.
- [21] Qian, C., W. Liu, H. Liu, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 15174–15186. 2024.
- [22] Chen, G., S. Dong, Y. Shu, et al. Autoagents: A framework for automatic agent generation. *arXiv preprint arXiv:2309.17288*, 2023.
- [23] Chen, C. Y., S. Saha, M. Bansal. Reconcile: Round-table conference improves reasoning via consensus among diverse LLMs, 2024.
- [24] Wang, J., J. WANG, B. Athiwaratkun, et al. Mixture-of-agents enhances large language model capabilities. In *The Thirteenth International Conference on Learning Representations*. 2025.
- [25] junyou li, Q. Zhang, Y. Yu, et al. More agents is all you need. *Transactions on Machine Learning Research*, 2024.
- [26] Pham, C., B. Liu, Y. Yang, et al. Let models speak ciphers: Multiagent debate through embeddings. In *The Twelfth International Conference on Learning Representations*. 2024.
- [27] Smit, A. P., N. Grinsztajn, P. Duckworth, et al. Should we be going MAD? a look at multi-agent debate strategies for LLMs. In *Forty-first International Conference on Machine Learning*. 2024.
- [28] Tang, Y., W. Su, Y. Zhou, et al. Augmenting multi-agent communication with state delta trajectory. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10230–10251. 2025.
- [29] Yang, E., L. Shen, G. Guo, et al. Model merging in llms, mllms, and beyond: Methods, theories, applications, and opportunities. *ACM Computing Surveys*, 58(8):1–41, 2026.
- [30] Li, W., Y. Peng, M. Zhang, et al. Deep model fusion: A survey. *arXiv preprint arXiv:2309.15698*, 2023.
- [31] Ilharco, G., M. T. Ribeiro, M. Wortsman, et al. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- [32] Wortsman, M., G. Ilharco, S. Y. Gadre, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. 2022.
- [33] Matena, M. S., C. A. Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- [34] Yadav, P., D. Tam, L. Choshen, et al. Ties-merging: Resolving interference when merging models. *Advances in neural information processing systems*, 36:7093–7115, 2023.
- [35] Yu, L., B. Yu, H. Yu, et al. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*. 2024.
- [36] Huang, C., Q. Liu, B. Y. Lin, et al. Lorahub: Efficient cross-task generalization via dynamic loRA composition. In *First Conference on Language Modeling*. 2024.
- [37] Wang, Z., S. He, K. Liu, et al. Instance-level dynamic loras composition for cross-task generalization. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 5699–5708. 2024.
- [38] Wang, H., B. Ping, S. Wang, et al. Lora-flow: Dynamic lora fusion for large language models in generative tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12871–12882. 2024.

- [39] Phang, J., Y. Mao, P. He, et al. Hypertuning: Toward adapting large language models without back-propagation. In *International Conference on Machine Learning*, pages 27854–27875. pmlr, 2023.
- [40] Charakorn, R., E. Cetin, S. Uesaka, et al. Doc-to-lora: Learning to instantly internalize contexts. *arXiv preprint arXiv:2602.15902*, 2026.
- [41] Team, T. H., et al. Hy-wu (part i): An extensible functional neural memory framework and an instantiation in text-guided image editing. *arXiv preprint arXiv:2603.07236*, 2026.
- [42] Su, J., M. Ahmed, Y. Lu, et al. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [43] Cobbe, K., V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [44] Hendrycks, D., C. Burns, S. Kadavath, et al. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [45] Gema, A. P., J. O. J. Leang, G. Hong, et al. Are we done with mmlu? In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5069–5096. 2025.
- [46] Hendrycks, D., C. Burns, S. Basart, et al. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [47] Liu, J., C. S. Xia, Y. Wang, et al. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems*, 36:21558–21572, 2023.
- [48] Chen, M., J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [49] Austin, J., A. Odena, M. Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [50] Yang, A., A. Li, B. Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [51] Guha, E., R. Marten, S. Keh, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.
- [52] Team, N. Sky-t1: Train your own o1 preview model within \$450. <https://novasky-ai.github.io/posts/sky-t1>, 2025. Accessed: 2025-01-09.
- [53] Xu, Z., Y. Liu, Y. Yin, et al. Kodcode: A diverse, challenging, and verifiable synthetic dataset for coding. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6980–7008. 2025.

470 A Limitations

471 While TFLOW offers an efficient alternative to text-based multi-agent communication, it also has
472 several limitations.

473 First, the communication channel is less interpretable than natural language. In text-based MAS,
474 intermediate messages can be inspected by humans to understand what each agent contributed. In
475 contrast, TFLOW transmits information through low-rank weight perturbations, whose semantic
476 content is difficult to directly interpret. This may complicate debugging, attribution, and safety
477 auditing, especially when the generated perturbations lead to unexpected receiver behavior.

478 Second, the performance gap on HumanEval+ indicates that TFLOW cannot fully recover the benefits
479 that text-based MAS may obtain from increased generation length. Text-based collaboration not
480 only transmits inter-agent information but also expands the overall reasoning and refinement budget

through explicit intermediate rationales, plans, and deliberation traces. These additional tokens can directly benefit tasks such as code synthesis, where longer solutions, iterative correction, and detailed implementation planning are often important. TFLOW avoids passing sender-written messages to the receiver and therefore removes much of this extra generation overhead. This improves efficiency, but it also means that TFLOW may not capture all gains arising from the longer generation process used by TextMAS. Future work may explore hybrid systems that combine concise textual messages with transient parameter perturbations to balance interpretability, generation budget, and efficiency.

B Additional Method Details

This appendix provides additional details for TFLOW that are omitted from the main text for brevity. We describe sender conditioning and layer aggregation, the parameter-generator architecture, sender fusion and transient receiver injection, the training objective, optimization algorithms, and computation cost.

B.1 Notation

We use N to denote the number of agents. Agents $A_1, \dots, A_{N-1}$ are senders and A_N is the receiver. All agents share the same frozen backbone parameters θ and are distinguished by role-specific prompts p_i . For a query q , sender A_i processes the prompted input (p_i, q) and produces hidden states $\{\mathbf{H}_i^{(l)}\}_{l=0}^L$, where

$$\mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}.$$

Here T_i is the sender sequence length and d is the hidden dimension of the frozen backbone.

We use L for the number of targeted receiver layers and M for the number of targeted linear modules per targeted layer. The frozen weight of module m in layer l is denoted by

$$\mathbf{W}_m^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times d_{\text{in}}^{(l,m)}}.$$

The LoRA rank is r , and α is the LoRA scaling coefficient. The parameter generator has hidden dimension d_{pg} and N_{pg} transformer blocks.

B.2 Sender Conditioning and Layer Aggregation

For each query, every sender runs one frozen forward pass on its prompted input. We retain hidden states from all backbone layers:

$$\{\mathbf{H}_i^{(l)}\}_{l=0}^L, \quad \mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}.$$

Since different layers encode complementary levels of abstraction, we aggregate them using learnable layer weights. Let λ_l be a scalar assigned to layer l , and let $\tau > 0$ be a temperature. The normalized layer weight is

$$\rho_l = \frac{\exp(\lambda_l/\tau)}{\sum_{l'=0}^L \exp(\lambda_{l'}/\tau)}. \quad (4)$$

The sender conditioning signal is then

$$\mathbf{C}_i = \sum_{l=0}^L \rho_l \mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}. \quad (5)$$

This construction lets the generator access information from both shallow and deep sender representations while preserving a fixed conditioning dimension.

Before entering the generator, the conditioning signal is projected to the generator hidden dimension:

$$\tilde{\mathbf{C}}_i = \text{Proj}_{\text{in}}(\mathbf{C}_i) \in \mathbb{R}^{T_i \times d_{\text{pg}}}. \quad (6)$$

The sender backbone remains frozen, and gradients do not update θ .

514 B.3 Parameter Generator

515 The parameter generator $\mathcal{H}_\psi$ maps each sender conditioning signal $\mathbf{C}_i$ to LoRA factors for every
 516 targeted receiver module. For each layer $l \in \{1, \dots, L\}$ and module $m \in \{1, \dots, M\}$, it outputs

$$\mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{r \times d_{\text{in}}^{(l,m)}}, \quad \mathbf{B}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times r}. \quad (7)$$

517 Thus, the corresponding low-rank update has shape

$$\mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times d_{\text{in}}^{(l,m)}}.$$

518 **Token-grid initialization.** The generator starts from a learnable token grid

$$\mathbf{Q}_{\text{grid}} \in \mathbb{R}^{L \times HW \times d_{\text{pg}}},$$

519 where $H = H_A + H_B$ separates tokens used for generating **A**- and **B**-factors, and W indexes
 520 module–rank slots. The projected sender condition $\tilde{\mathbf{C}}_i$ initializes this grid through cross-attention:

$$\mathbf{Z}_i^{(0)} = \text{CA}(\mathbf{Q}_{\text{grid}}, \tilde{\mathbf{C}}_i, \tilde{\mathbf{C}}_i) + \mathbf{T}_{\text{layer}} + \mathbf{T}_{\text{slot}}, \quad (8)$$

521 where $\mathbf{T}_{\text{layer}}$ encodes the target receiver-layer index and $\mathbf{T}_{\text{slot}}$ encodes module and rank-slot posi-
 522 tions.

523 **Multi-axis transformer trunk.** The initialized grid is refined by N_{pg} transformer blocks. Each
 524 block contains cross-layer self-attention, intra-layer self-attention, conditioning cross-attention, and a
 525 feed-forward network. For block t , we write

$$\mathbf{U}_i^{(t)} = \mathbf{Z}_i^{(t)} + \text{SA}_L(\text{LN}(\mathbf{Z}_i^{(t)})), \quad (9)$$

$$\mathbf{V}_i^{(t)} = \mathbf{U}_i^{(t)} + \text{SA}_{HW}(\text{LN}(\mathbf{U}_i^{(t)})), \quad (10)$$

$$\mathbf{Y}_i^{(t)} = \mathbf{V}_i^{(t)} + \text{CA}(\text{LN}(\mathbf{V}_i^{(t)}), \tilde{\mathbf{C}}_i, \tilde{\mathbf{C}}_i), \quad (11)$$

$$\mathbf{Z}_i^{(t+1)} = \mathbf{Y}_i^{(t)} + \text{FFN}(\text{LN}(\mathbf{Y}_i^{(t)})). \quad (12)$$

526 Here, SA_L attends across targeted receiver layers while batching over the HW axis, and SA_{HW}
 527 attends within each layer across module–rank slots. The conditioning cross-attention re-grounds
 528 the parameter tokens in the sender representations at every block. RoPE is applied along the active
 529 attention axis.

530 **Detokenization into LoRA factors.** After the final generator block, the grid is split along the H
 531 axis into **A**-tokens and **B**-tokens:

$$\mathbf{Z}_i^{(N_{\text{pg}})} = [\mathbf{Z}_{i,A}, \mathbf{Z}_{i,B}].$$

532 Two linear heads project these tokens into parameter slices:

$$\mathbf{T}_{i,A} = h_A(\mathbf{Z}_{i,A}), \quad \mathbf{T}_{i,B} = h_B(\mathbf{Z}_{i,B}). \quad (13)$$

533 The slices are rearranged according to layer, module, and rank-slot indices to produce

$$\left\{ \mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)} \right\}_{l=1, m=1}^{L, M}.$$

534 All generator parameters are shared across senders. Sender-specific LoRA factors arise solely from
 535 sender-specific conditioning signals $\mathbf{C}_i$.

536 B.4 Sender Fusion and Transient Injection

537 When multiple senders are active, TFLOW fuses their generated updates at the perturbation level.
 538 This is important because fusing **A**- and **B**-factors separately would create unintended cross-sender
 539 products. Specifically,

$$\left(\sum_i \gamma_i \mathbf{B}_i \right) \left(\sum_j \gamma_j \mathbf{A}_j \right) = \sum_{i,j} \gamma_i \gamma_j \mathbf{B}_i \mathbf{A}_j,$$

540 which includes terms $\mathbf{B}_i \mathbf{A}_j$ for $i \neq j$. We therefore fuse complete low-rank perturbations $\mathbf{B}_i \mathbf{A}_i$.

541 The sender-fusion gate predicts one scalar score for each sender:

$$s_i = g_{\mathbf{w}}(\mathbf{C}_i). \quad (14)$$

542 Scores are normalized across active senders:

$$\gamma_i = \frac{\exp(s_i)}{\sum_{j=1}^{N-1} \exp(s_j)}. \quad (15)$$

543 The fused transient update for module m in layer l is

$$\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}. \quad (16)$$

544 For a receiver activation matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{in}}^{(l,m)}}$, the patched linear map is

$$\mathbf{X} \mapsto \mathbf{X}(\mathbf{W}_m^{(l)})^\top + \mathbf{X}(\Delta \mathbf{W}_m^{(l)})^\top. \quad (17)$$

545 The dense matrix $\Delta \mathbf{W}_m^{(l)}$ need not be explicitly materialized. The additive branch can be computed
546 in low-rank form:

$$\mathbf{X}(\Delta \mathbf{W}_m^{(l)})^\top = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i (\mathbf{X}(\mathbf{A}_{i,m}^{(l)})^\top) (\mathbf{B}_{i,m}^{(l)})^\top. \quad (18)$$

547 The frozen weight tensor $\mathbf{W}_m^{(l)}$ is never overwritten. The update exists only during the current receiver
548 forward pass and is removed immediately afterward.

549 B.5 Training Objective

550 The trainable parameter set is

$$\Theta = \{\psi, \text{Proj}_{\text{in}}, \{\lambda_l\}_{l=0}^{L_{\text{total}}}, \mathbf{w}\},$$

551 where ψ denotes the parameter-generator weights, Proj_{in} denotes the conditioning projection, $\{\lambda_l\}$
552 are the layer-aggregation scalars, and $\mathbf{w}$ denotes the fusion-gate parameters. The frozen backbone
553 parameters θ are excluded from Θ .

554 **Task loss.** For a training tuple (x_q, t, σ) , where x_q is the query-side input, t is the target completion,
555 and σ is the data-source identifier, the task loss is the masked next-token cross-entropy over completion
556 tokens:

$$\mathcal{L}_{\text{task}}(\Theta; x_q, t) = - \sum_{u \in \mathcal{I}_{\text{ans}}} \log p_{\theta, \Theta}(t_u \mid t_{<u}, x_q), \quad (19)$$

557 where $\mathcal{I}_{\text{ans}}$ denotes answer-token positions. The distribution $p_{\theta, \Theta}$ is computed by the frozen receiver
558 equipped with the transient updates produced by Θ .

559 **Flattened generated update.** To define the diversity regularizer, we flatten all fused updates for
560 the current input:

$$v_{\Theta}(x_q) = \text{vec} \left(\left\{ \Delta \mathbf{W}_m^{(l)}(x_q) \right\}_{l=1, m=1}^{L, M} \right). \quad (20)$$

561 In implementation, this vector can be represented either by explicitly flattened dense offsets or by an
562 equivalent flattened low-rank representation, as long as the same representation is used consistently
563 for the current vector and the cache.

564 **Cache-based diversity regularizer.** For each data source σ , we maintain a stop-gradient cache
565 vector $v_{\text{prev}}^{(\sigma)}$, corresponding to the generated update from the previous optimization step involving
566 that source. The diversity regularizer is

$$\mathcal{L}_{\text{div}}(\Theta; x_q) = \cos^2(v_{\Theta}(x_q), v_{\text{prev}}^{(\sigma(x_q))}). \quad (21)$$

567 This objective penalizes both parallel and anti-parallel collapse and is minimized when the current
568 update is orthogonal to the cached update. Since $v_{\text{prev}}^{(\sigma)}$ is detached, the regularizer does not introduce
569 an additional forward pass.

570 The total objective is

$$\mathcal{L}(\Theta) = \mathbb{E}_{(x_q, t, \sigma) \sim \mathcal{D}} [\mathcal{L}_{\text{task}}(\Theta; x_q, t) + \lambda_{\text{div}} \mathcal{L}_{\text{div}}(\Theta; x_q)]. \quad (22)$$

571 B.6 Training Algorithm

Algorithm 1 Training TFLOW

Require: Frozen backbone f_θ ; trainable parameters Θ ; sender prompts $\{p_i\}_{i=1}^{N-1}$; receiver prompt p_N ; dataset $\mathcal{D}$; diversity weight λ_{div} .

- 1: Initialize diversity cache $\{v_{\text{prev}}^{(\sigma)}\}$ for each data source σ .
- 2: **for** each minibatch $\mathcal{B} \subset \mathcal{D}$ **do**
- 3: Initialize minibatch loss $\mathcal{L}_{\mathcal{B}} \leftarrow 0$.
- 4: **for** each example $(x_q, t, \sigma) \in \mathcal{B}$ **do**
- 5: **for** each sender $i = 1, \dots, N-1$ **do**
- 6: Run frozen sender forward on (p_i, x_q) and collect $\{\mathbf{H}_i^{(l)}\}_{l=0}^{L_{\text{total}}}$.
- 7: Aggregate hidden states into $\mathbf{C}_i$ using Equation (5).
- 8: Generate LoRA factors $\{\mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)}\}_{l,m} \leftarrow \mathcal{H}_\psi(\mathbf{C}_i)$.
- 9: **end for**
- 10: Compute gate weights $\gamma_i = \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i))$.
- 11: Fuse sender updates into $\Delta \mathbf{W}_m^{(l)}$ for all targeted modules using Equation (16).
- 12: Temporarily patch the targeted receiver modules.
- 13: Run the frozen receiver on (p_N, x_q, t) and compute $\mathcal{L}_{\text{task}}$ over completion tokens.
- 14: Form $v_\Theta(x_q)$ and compute $\mathcal{L}_{\text{div}}$ using the stop-gradient cache $v_{\text{prev}}^{(\sigma)}$.
- 15: Accumulate $\mathcal{L}_{\mathcal{B}} \leftarrow \mathcal{L}_{\mathcal{B}} + \mathcal{L}_{\text{task}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}$.
- 16: Remove the temporary receiver patches.
- 17: **end for**
- 18: Update Θ using gradients of $\mathcal{L}_{\mathcal{B}}$.
- 19: Update each used cache entry $v_{\text{prev}}^{(\sigma)} \leftarrow \text{stopgrad}(v_\Theta(x_q))$.
- 20: **end for**

572 B.7 Inference Algorithm

573 At inference time, TFLOW follows the same sender-conditioning and parameter-generation process,
 574 but omits the task loss, diversity regularizer, and cache update. The generated perturbation is used
 575 only during receiver decoding.

Algorithm 2 Inference with TFLOW

Require: Frozen backbone f_θ ; trained parameters Θ ; sender prompts $\{p_i\}_{i=1}^{N-1}$; receiver prompt p_N ; query q .

- 1: **for** each sender $i = 1, \dots, N-1$ **do**
- 2: Run frozen sender forward on (p_i, q) and collect $\{\mathbf{H}_i^{(l)}\}_{l=0}^{L_{\text{total}}}$.
- 3: Aggregate hidden states into $\mathbf{C}_i$.
- 4: Generate LoRA factors $\{\mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)}\}_{l,m} \leftarrow \mathcal{H}_\psi(\mathbf{C}_i)$.
- 5: **end for**
- 6: Compute $\gamma_i = \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i))$.
- 7: Fuse updates into $\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}$.
- 8: Temporarily patch the targeted receiver modules.
- 9: Decode the final answer using receiver A_N conditioned on (p_N, q) .
- 10: Remove all temporary patches.

576 B.8 Computation Cost

577 The additional cost of TFLOW beyond a standard receiver forward pass comes from sender forwarding,
 578 parameter generation, and transient LoRA injection.

579 **Sender forward passes.** Let $C_{\text{bb}}(T)$ denote the cost of one frozen backbone forward pass with
 580 sequence length T . The $N - 1$ senders incur

$$C_{\text{sender}} = \sum_{i=1}^{N-1} C_{\text{bb}}(T_i). \quad (23)$$

581 These sender passes are independent and can be parallelized. They also do not require autoregressive
 582 decoding; each sender only produces hidden states for conditioning.

583 **Parameter generation.** Let $S = HW$ be the number of parameter tokens per targeted layer. Each
 584 generator block applies cross-layer attention, intra-layer attention, conditioning cross-attention, and
 585 an FFN. For sender i , the cross-layer attention costs $O(SL^2d_{\text{pg}})$, the intra-layer attention costs
 586 $O(LS^2d_{\text{pg}})$, the conditioning cross-attention costs $O(LST_id_{\text{pg}})$, and the FFN costs $O(LSd_{\text{pg}}^2)$.
 587 Therefore,

$$C_{\text{gen}}^{(i)} = O(N_{\text{pg}} [SL^2d_{\text{pg}} + LS^2d_{\text{pg}} + LST_id_{\text{pg}} + LSd_{\text{pg}}^2]), \quad (24)$$

588 and across all senders,

$$C_{\text{gen}} = \sum_{i=1}^{N-1} C_{\text{gen}}^{(i)}. \quad (25)$$

589 **Transient LoRA injection.** For a targeted linear module with receiver sequence length T , input
 590 dimension $d_{\text{in}}^{(l,m)}$, output dimension $d_{\text{out}}^{(l,m)}$, and LoRA rank r , the low-rank branch in Equation (18)
 591 costs

$$O((N-1)Tr(d_{\text{in}}^{(l,m)} + d_{\text{out}}^{(l,m)})). \quad (26)$$

592 This is small compared with the dense projection cost

$$O(Td_{\text{in}}^{(l,m)}d_{\text{out}}^{(l,m)})$$

593 when $r \ll \min(d_{\text{in}}^{(l,m)}, d_{\text{out}}^{(l,m)})$. Summing over all targeted modules gives

$$C_{\text{inj}} = O\left((N-1)Tr \sum_{l=1}^L \sum_{m=1}^M (d_{\text{in}}^{(l,m)} + d_{\text{out}}^{(l,m)})\right). \quad (27)$$

594 **Overall overhead.** The total additional cost is

$$C_{\text{extra}} = C_{\text{sender}} + C_{\text{gen}} + C_{\text{inj}}. \quad (28)$$

595 The sender and generator costs scale linearly with the number of active senders $N - 1$, and the
 596 injection cost scales linearly with both $N - 1$ and the LoRA rank r . In practice, sender forward passes
 597 and per-sender generation are independent and can be batched or parallelized, reducing wall-clock
 598 overhead.

599 C Ethics, Broader Impacts, and Asset Documentation

600 C.1 Code of Ethics

601 This work conforms to the NeurIPS Code of Ethics. The study is based on existing LLM backbones,
 602 standard public benchmarks, and algorithmic evaluation of multi-agent LLM collaboration. It does
 603 not involve crowdsourcing, user studies, personally identifiable information, or other research with
 604 human subjects. We do not release pre-trained language models, image generators, scraped datasets,
 605 or other artifacts that are specifically designed to enable harmful use.

606 C.2 Broader Impacts

607 The proposed method aims to improve the efficiency of multi-agent LLM collaboration by replacing
 608 natural-language inter-agent messages with transient weight-space perturbations. Potential positive
 609 impacts include reduced generated-token usage, lower latency, smaller KV-cache footprint, and lower

610 compute cost for collaborative LLM systems. These efficiency gains may make multi-agent reasoning
611 systems more accessible to researchers and practitioners with limited computational resources.

612 At the same time, more efficient LLM-agent systems may also reduce the cost of undesirable
613 applications, including automated generation of misleading content, misuse of coding agents, or
614 large-scale deployment of unreliable automated decision-making systems. Our method does not
615 introduce new generative capabilities beyond those of the underlying LLM backbones, but it may
616 make some existing capabilities cheaper to use in multi-agent settings. We therefore recommend that
617 deployments of systems based on this approach follow the safety policies, access controls, monitoring
618 practices, and usage restrictions appropriate for the underlying LLMs and application domains.

619 **C.3 Existing Assets and Licenses**

620 Our experiments use existing LLM backbones, benchmark datasets, and software libraries. We
621 cite the original sources of the models, datasets, and libraries used in the paper. We follow the
622 corresponding licenses and terms of use for all existing assets. The model checkpoints are used only
623 under their permitted research or open-use terms, and the benchmark datasets are used for evaluation
624 according to their respective licenses or usage policies.

625 The main existing assets used in this work include the Qwen3-4B backbone, evaluation benchmarks
626 such as GSM8K and HumanEval+, and standard machine learning libraries such as PyTorch, Hugging
627 Face Transformers, and LoRA-related implementations. We provide versions, access instructions,
628 and license information for these assets in the supplemental material and released code package,
629 where applicable.

630 **C.4 New Assets**

631 We release new assets associated with TFLOW, including code for weight-space multi-agent col-
632 laboration, the receiver-specific parameter generator, experiment configuration files, and scripts for
633 reproducing the main results. The released assets are documented with environment specifications,
634 dependency versions, data preparation instructions, and commands for running the main experiments.

635 The released assets are intended for research and reproducibility. They do not contain personally
636 identifiable information, crowdsourced annotations, or newly scraped datasets. The released package
637 is anonymized when required by the review process. The final version will include license information,
638 citation instructions, and documentation describing the intended use and limitations of the released
639 code and trained components.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The main claims and contributions are stated in the abstract and Section 1, and are supported by the theoretical analysis in Section 3 and experiments in Section 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper includes a separate Limitations section in A that discusses key limitations of the proposed method.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present theoretical results; its contributions are empirical and methodological.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide implementation details needed to reproduce the main experiments in Sections 3 and Appendix B. Additional details and scripts are provided in the supplemental material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide anonymized code and data in the supplemental material, including environment specifications, dependency versions, and commands to reproduce the main results in Tables 1.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sections 3 and 4 describe the datasets, model architectures, hyperparameters, training schedule, and evaluation metrics. Full configuration files are included in the supplemental material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Due to computational cost, the largest-scale experiments were run once.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4 reports the GPU type and runtime per experiment for the reported experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. The work uses existing LLM backbones and standard benchmarks, does not involve human subjects, and does not release high-risk models or datasets intended for misuse. See Appendix C.1.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses positive impacts such as reduced token usage, latency, KV-cache footprint, and compute cost, as well as potential negative impacts such as cheaper misuse or deployment of LLM-based multi-agent systems. See Appendix C.2.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release models or datasets that pose a high risk of misuse.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper credits the original creators of existing LLM backbones, benchmark datasets, and software libraries, and documents their licenses, versions, and terms of use. See Appendix C.3.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The released code, trained components, and experimental configurations for TFLOW are documented with environment specifications, usage instructions, and reproduction commands. See Appendix C.4.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing, user studies, or other research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

952 Answer: [N/A]

953 Justification: The paper does not involve crowdsourcing or research with human subjects.

954 Guidelines:

- 955 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 956 with human subjects.
- 957 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 958 may be required for any human subjects research. If you obtained IRB approval, you
- 959 should clearly state this in the paper.
- 960 • We recognize that the procedures for this may vary significantly between institutions
- 961 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 962 guidelines for their institution.
- 963 • For initial submissions, do not include any information that would break anonymity (if
- 964 applicable), such as the institution conducting the review.

965 **16. Declaration of LLM usage**

966 Question: Does the paper describe the usage of LLMs if it is an important, original, or

967 non-standard component of the core methods in this research? Note that if the LLM is used

968 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

969 scientific rigor, or originality of the research, declaration is not required.

970 Answer: [Yes]

971 Justification: LLMs are a central component of the proposed method. The paper describes

972 the use of frozen LLM backbones as sender and receiver agents, where sender hidden states

973 are mapped into query-specific LoRA perturbations for the receiver, in Sections 1 and 3.

974 Guidelines:

- 975 • The answer [N/A] means that the core method development in this research does not
- 976 involve LLMs as any important, original, or non-standard components.
- 977 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 978 be described.